

Comparative Analysis of Variable Ordering Heuristics in Backtracking-Based Sudoku Solving

Alfie Jenkins

School of Computing

Falmouth University

Cornwall, United Kingdom

aj313798@falmouth.ac.uk

Abstract—This paper presents a comparative evaluation of three variable ordering heuristics for solving Sudoku puzzles using a backtracking algorithm. We compare sequential cell selection, random cell selection, and the Minimum Remaining Values (MRV) heuristic across 200 standard 9×9 Sudoku puzzles. Results demonstrate that MRV achieves zero backtracks on all test puzzles, while sequential selection averages 95.70 backtracks (median 60) and random selection averages 33,216 backtracks (median 14,517). Mann-Whitney U tests confirm all pairwise differences are highly significant ($p < 0.001$). Despite MRV’s higher per-iteration computational cost, it demonstrates superior overall efficiency through dramatic reduction in search space exploration. These findings confirm the importance of intelligent variable ordering in constraint satisfaction problems.

Index Terms—constraint satisfaction, backtracking, heuristics, Sudoku, variable ordering, MRV

I. INTRODUCTION

Constraint Satisfaction Problems (CSPs) form a fundamental class of artificial intelligence problems with applications spanning scheduling, resource allocation, and puzzle solving. The performance of CSP solvers depends critically on search strategies, particularly the order in which variables are assigned values. This paper investigates how variable ordering heuristics affect backtracking algorithm performance in the context of Sudoku puzzle solving.

Sudoku serves as an ideal benchmark for CSP research due to its well-defined constraints, finite solution space, and scalability. While a 9×9 Sudoku grid has approximately 6.67×10^{21} possible configurations, intelligent search strategies can solve puzzles in milliseconds.

This work compares three variable ordering approaches: sequential (baseline), random (control), and Minimum Remaining Values (MRV). We measure backtracking frequency and execution time across 50 puzzles to quantify the impact of heuristic choice on solver efficiency. Understanding heuristic efficiency extends beyond puzzles: CSP techniques underpin real-world scheduling, logistics, and resource allocation systems where solver performance affects operational cost, fairness, and energy consumption.

The remainder of this paper is organized as follows: Section II reviews related work on CSPs and Sudoku solving; Section III describes our methodology; Section IV presents results; Section V discusses findings; and Section VI concludes with implications and future work.

II. BACKGROUND AND RELATED WORK

A. Constraint Satisfaction Problems

A CSP consists of three components: a set of variables $V = \{V_1, V_2, \dots, V_n\}$, domains $D = \{D_1, D_2, \dots, D_n\}$ specifying possible values for each variable, and constraints C that restrict valid value combinations [1]. Finding a solution requires assigning values to all variables such that all constraints are satisfied.

CSP solving is NP-complete in general [2], making algorithm efficiency crucial. Backtracking search with constraint propagation remains the standard approach for exact solutions.

B. Sudoku as a CSP

Sudoku can be formalized as a CSP with 81 variables (cells), each with domain $\{1, 2, \dots, 9\}$, and 27 all-different constraints (9 rows, 9 columns, 9 boxes) [3]. The problem is to assign values satisfying all constraints.

While general $n \times n$ Sudoku is NP-complete [2], standard 9×9 puzzles have finite complexity and can be solved efficiently with appropriate algorithms.

C. Backtracking Search

Backtracking systematically explores the search space by recursively assigning values and undoing assignments when constraints are violated [1]. The algorithm’s efficiency depends on minimizing failed branches through intelligent variable and value ordering.

D. Variable Ordering Heuristics

The Minimum Remaining Values (MRV) heuristic, also called the “most constrained variable” or “fail-first” heuristic, selects the variable with the fewest legal values remaining [4]. By choosing highly constrained variables first, MRV detects failures early and prunes the search tree aggressively.

Norvig demonstrated MRV’s effectiveness in Sudoku solving, integrating it with constraint propagation [4]. Russell and Norvig note that MRV consistently reduces search tree size across CSP domains by prioritising variables most likely to cause early failure [1].

When multiple cells share the minimum remaining value count, a tie-breaking strategy is required. The degree heuristic selects the variable involved in the most constraints with

unassigned neighbours [1]. Our implementation breaks ties arbitrarily; degree-based tie-breaking is left as a future extension.

Value ordering — the sequence in which candidate values are tried for a given cell — is a complementary dimension to variable ordering. Russell and Norvig describe the Least Constraining Value (LCV) heuristic, which prefers values that rule out the fewest options for neighbouring variables [1]. Our work focuses on variable ordering, leaving value ordering as a natural extension.

E. Constraint Propagation

Constraint propagation techniques reduce variable domains during search, pruning the search space before backtracking occurs. Forward checking removes from neighbouring domains any values that conflict with a new assignment [8]. Arc consistency (AC-3) extends this by enforcing constraints between all pairs of variables, ensuring that for every value in one variable’s domain there exists a compatible value in each neighbour’s domain [7].

Norvig’s Sudoku solver combines MRV with constraint propagation, achieving greater efficiency than backtracking alone by eliminating infeasible values before they are explored [4]. Our implementation deliberately excludes propagation to isolate the effect of variable ordering on search performance; incorporating propagation is left as a future extension.

III. METHODOLOGY

A. Implementation

A backtracking solver was implemented in Python 3.14 following standard recursive approaches [1]. The core algorithm:

```

find next empty cell using heuristic
if no empty cells then return solution found end if
for each value  $v \in \{1 \dots 9\}$  do
  if  $v$  is valid at cell then
    assign  $v$  to cell
    if recursive solve succeeds then return success end if
    remove  $v$  from cell; increment backtrack counter
  end if
end for
return failure

```

Cell validity checking ensures no duplicate values in the cell’s row, column, or 3×3 box.

B. Variable Ordering Heuristics

Three heuristics determine the next empty cell:

Sequential: Cells are selected in row-major order. This baseline approach requires no computation but provides no intelligent ordering.

Random: A cell is selected uniformly at random from all empty cells. This control condition tests whether any ordering outperforms chance.

MRV: For each empty cell, count valid remaining values. Select the cell with minimum count. Ties are broken arbitrarily. This implements the fail-fast principle: constrained cells are more likely to cause failures, so testing them early prunes the search space.

C. Experimental Design

Dataset: The first 200 puzzles from the Kaggle Sudoku dataset [5] were used. Puzzles are represented as 81-character strings with ‘0’ for empty cells. The dataset is released under a CC0 public domain licence, permitting unrestricted academic use; no personally identifiable data is present.

Procedure: A fixed random seed (42) was set before all runs to ensure reproducibility of the random heuristic. Each puzzle was solved once with each heuristic. The board state was reset between runs to ensure identical starting conditions. For each run, we measured:

- Execution time (seconds)
- Number of backtracks
- Solution correctness (verified against provided solutions)

Platform: Experiments ran on Arch Linux with Python 3.14, pandas 2.0, and numpy 1.24. Code is available at <https://github.com/AlfieMoon21/sudoku-heuristics>.

IV. RESULTS

Tables I and II present performance and statistical test results. Figure 1 shows average backtracks per heuristic.

TABLE I
PERFORMANCE SUMMARY STATISTICS (N=200)

Heuristic	Mean BT	Median BT	Mean Time (s)	Max BT
Sequential	95.70	60	0.000421	640
Random	33,216	14,517	0.185786	582,024
MRV	0	0	0.004142	0

TABLE II
MANN-WHITNEY U TEST RESULTS (TWO-SIDED)

Comparison	U statistic	p-value
Sequential vs Random	267	2.58×10^{-65}
Sequential vs MRV	40,000	2.35×10^{-76}
Random vs MRV	40,000	2.36×10^{-76}

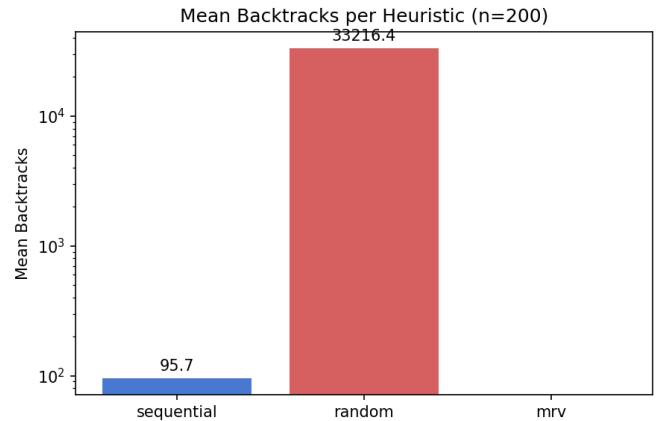


Fig. 1. Average backtracks per heuristic

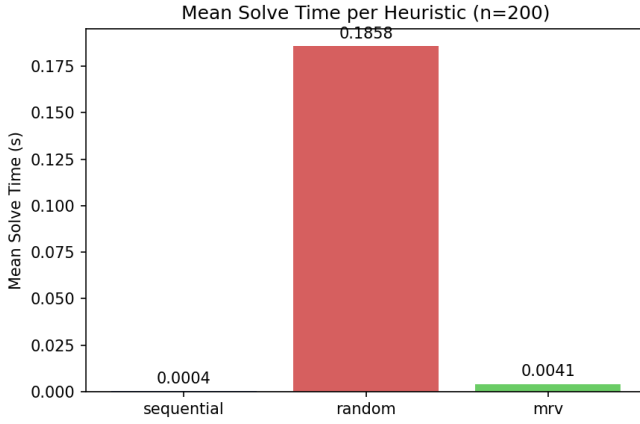


Fig. 2. Average solving time per heuristic

MRV achieved zero backtracks on all 200 puzzles, indicating perfect variable selection. This result should be interpreted in the context of dataset difficulty: all puzzles are standard, well-formed 9×9 grids with unique solutions. Harder instances — or adversarially constructed puzzles with minimal givens — may still require backtracking under pure MRV without constraint propagation [2].

Sequential selection averaged 95.70 backtracks (median 60, $\sigma=103.69$). The gap between mean and median indicates a right-skewed distribution: many puzzles are solved with few backtracks, while a tail of structurally challenging puzzles inflates the mean. Figure 3 shows this distribution.

Random selection performed worst, averaging 33,216 backtracks (median 14,517, $\sigma=54,529$) with maximum 582,024. The 347-fold mean increase over sequential, and 242-fold median increase, confirms that variable ordering dramatically impacts efficiency.

Despite solving puzzles without backtracking, MRV’s mean execution time (0.00414s) was $9.8\times$ slower than sequential (0.000421s), reflecting the per-iteration overhead of computing remaining values for all empty cells.

However, random selection’s 0.186s average time demonstrates that backtracking cost dominates for inefficient heuristics. From a sustainability perspective, this 440× difference in solve time between random and MRV translates directly to energy cost at deployment scale: a scheduling system invoking a solver thousands of times per day would incur substantial unnecessary energy expenditure through poor heuristic choice.

A. Statistical Significance

Mann-Whitney U tests (Table II) confirm that all pairwise differences are highly significant ($p < 0.001$). A non-parametric test was chosen because backtrack distributions are strongly right-skewed. The U statistic of 40,000 for both Sequential vs MRV and Random vs MRV equals $n_1 \times n_2 = 200 \times 200$, the theoretical maximum, confirming that every MRV run produced strictly fewer backtracks than any Sequential or Random run.

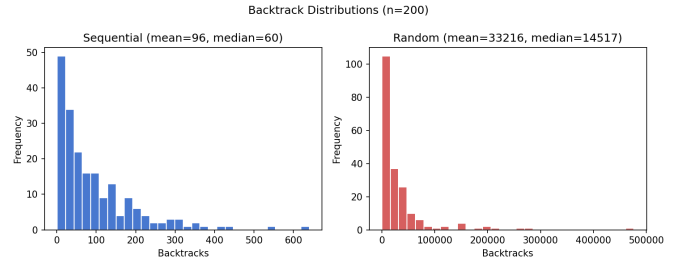


Fig. 3. Distribution of backtracks per heuristic across 200 puzzles. MRV omitted (all zero).

B. Puzzle Difficulty Correlation

Figure 4 plots empty cell count against sequential backtracks. Pearson correlation yielded $r = 0.043$ ($p = 0.546$), indicating no significant linear relationship. This is a notable finding: puzzle difficulty as measured by given-cell count does not predict where sequential struggles. Performance appears to depend on the *spatial arrangement* of given cells rather than their count.

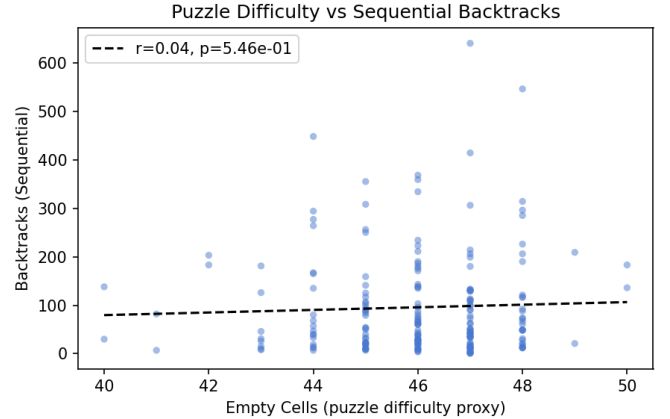


Fig. 4. Empty cell count vs sequential backtracks ($r = 0.043$, $p = 0.546$). No significant correlation.

V. DISCUSSION

A. Why MRV Achieves Zero Backtracks

MRV’s perfect performance stems from the fail-fast principle. By always selecting the most constrained cell, MRV either:

- 1) Finds a cell with one valid value (forced move)
- 2) Identifies an unsolvable state immediately

For valid Sudoku puzzles, constraint propagation through MRV ordering creates a sequence of forced moves, eliminating the need for speculative assignments.

B. Computational Trade-offs

MRV incurs $O(n \cdot k)$ overhead per cell selection, where n is empty cells and k is domain size (9). For a 9×9 Sudoku, this adds approximately 81 constraint checks per iteration.

Sequential ordering has $O(1)$ cell selection but explores exponentially more search branches. Our results show MRV’s linear overhead is vastly preferable to exponential search.

This aligns with the broader CSP literature: intelligent heuristics reduce total search cost despite higher per-step overhead [1].

C. Implications for CSP Solving

These results generalize beyond Sudoku. Variable ordering is crucial for CSP solver performance. The 347-fold mean difference between random and sequential backtracks, and MRV’s complete elimination of backtracking, demonstrates that heuristic choice is not a minor optimization but a fundamental algorithmic decision.

D. Spatial Arrangement and Puzzle Difficulty

The non-significant difficulty correlation ($r = 0.043$, $p = 0.546$) reveals an important structural property of the sequential heuristic. Given-cell count is the most natural proxy for puzzle difficulty, yet it explains essentially none of the variance in sequential backtracks. This suggests that the sequential heuristic’s performance is governed by the *topology* of the constraint graph at solve time: whether early row-major cells happen to be highly constrained relative to their neighbours, rather than how many cells are empty overall. A puzzle with 51 empty cells arranged so that the first few empty cells are in sparse regions may require far more backtracking than a puzzle with 55 empty cells whose first empty cells are tightly constrained. This finding motivates future work on structural puzzle metrics beyond simple cell counts.

E. Limitations

Our study focuses on standard 9×9 Sudoku from a single dataset. All 200 puzzles are drawn from the same distribution of difficulty and construction method; results may differ for puzzles generated by different methods, for harder puzzles with fewer givens, or for larger grid variants. Additionally, we examined only three heuristics; hybrid approaches combining MRV with degree heuristics or value ordering strategies merit investigation.

VI. CONCLUSION

This paper demonstrates that variable ordering heuristics profoundly impact backtracking algorithm performance. MRV eliminates backtracking entirely for standard Sudoku puzzles, while random selection increases mean backtracks 347-fold compared to sequential ordering across 200 test puzzles.

Despite MRV’s $9.8\times$ computational overhead per iteration, it solves puzzles $44\times$ faster than random selection by avoiding unnecessary search. Mann-Whitney U tests confirm all pairwise differences are highly significant ($p < 0.001$). This confirms that intelligent variable ordering is essential for efficient CSP solving.

A. Legal, Social, Ethical, Professional, and Sustainability Considerations

Legal. CSP solvers deployed in safety-critical domains — air traffic scheduling, medical resource allocation, or financial constraint systems — may be subject to regulatory requirements on algorithmic transparency and auditability. Selecting an inefficient heuristic that causes solver timeouts in such a system could constitute professional negligence. Developers have a duty to document algorithm choices and their known limitations.

Social. Heuristic CSP solvers are used in workforce scheduling, exam timetabling, and logistics. A suboptimal variable ordering (analogous to our sequential or random heuristics) may produce solutions that are technically valid but unfair in practice — for example, consistently assigning unsociable hours to particular workers. The 347-fold performance gap between random and sequential ordering observed here illustrates how algorithm choice has real distributional consequences when applied at scale.

Ethical. Algorithm transparency is an ethical obligation. Users of automated scheduling or configuration systems should be informed whether the system is guaranteed to find an optimal solution or is applying a heuristic that may not. Presenting a heuristic result as optimal without disclosure is misleading. This is particularly relevant as MRV, while highly effective on standard Sudoku, may not achieve zero backtracks on harder or larger CSP instances.

Professional. Professional engineers selecting CSP solvers must weigh the trade-offs demonstrated in this work. Sequential selection is simple to implement and audit, while MRV requires more careful implementation and testing. Choosing the wrong heuristic for a production system — particularly one where solver performance degrades under load — is a foreseeable professional risk. The ACM Code of Ethics [6] requires practitioners to act competently and to understand the limitations of the techniques they deploy.

Sustainability. Computational efficiency has direct energy implications. The random heuristic’s 0.186s average solve time is 440 times slower than MRV. At scale — thousands of solver invocations per day in a scheduling system — this difference translates to significant unnecessary energy consumption. Choosing efficient heuristics such as MRV is a concrete sustainability decision, reducing the carbon footprint of constraint-based software.

B. Future Work

Future research directions include:

- Testing on larger Sudoku variants and other CSP domains
- Investigating hybrid heuristics (MRV with degree heuristic tie-breaking)
- Combining intelligent variable and value ordering
- Analyzing puzzle features that predict heuristic performance

CSP solvers incorporating MRV-style heuristics could improve efficiency in scheduling systems, configuration tools,

and other constraint-based applications, reducing computational costs while maintaining solution quality.

REFERENCES

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
- [2] T. Yato and T. Seta, "Complexity and completeness of finding another solution and its application to puzzles," *IEICE Trans. Fundamentals*, vol. E86-A, no. 5, pp. 1052–1060, 2003.
- [3] H. Simonis, "Sudoku as a constraint problem," in *CP Workshop on Modeling and Reformulating Constraint Satisfaction Problems*, vol. 12, pp. 13–27, 2005.
- [4] P. Norvig, "Solving every Sudoku puzzle," 2006. [Online]. Available: <https://norvig.com/sudoku.html>
- [5] B. Park, "Sudoku dataset," Kaggle, 2017. [Online]. Available: <https://www.kaggle.com/datasets/bryanpark/sudoku>
- [6] ACM, "ACM Code of Ethics and Professional Conduct," 2018. [Online]. Available: <https://www.acm.org/code-of-ethics>
- [7] A.K. Mackworth, "Consistency in networks of relations," *Artificial Intelligence*, vol. 8, no. 1, pp. 99–118, 1977.
- [8] R.M. Haralick and G.L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980.